

C++ 分布式 RPC 项目升级全流程规划

从教程型项目到可写入简历、可经受面试深挖的工程化项目

适用方向：C++ 后台开发 / 服务端开发 / AI Infra 边缘后台方向

版本：v1.0 日期：2026-06-26

目录

1. 项目定位与最终成品边界
2. 总体架构与模块拆分
3. 开发前准备与工程结构
4. Milestone 1：可运行基线与网络层解耦
5. Milestone 2：自定义协议与 RpcCodec
6. Milestone 3：Protobuf 元信息与服务端分发
7. Milestone 4：客户端同步调用
8. Milestone 5：异步调用、request_id 与超时管理
9. Milestone 6：ZooKeeper 服务注册与发现
10. Milestone 7：协议错误处理与异常请求防护
11. Milestone 8：文件分块传输业务
12. Milestone 9：内存池与对象复用优化
13. Milestone 10：性能压测与指标体系
14. Milestone 11：文档、简历与面试材料
15. 推荐时间线与优先级
16. 知识准备清单
17. 最终验收清单
18. 第一天可执行任务

1. 项目定位与最终成品边界

当前项目基础是 Muduo/Reactor 网络模型、Protobuf、ZooKeeper 和 Glog 组成的 RPC 服务注册与调用系统。主要问题不是技术栈错误，而是简历表达和工程深度容易被判断为常见复现项目。升级目标是把项目从“能调用”提升为“有协议健壮性、有异步调用、有业务验证、有内存优化、有压测数据”的工程项目。

最终项目一句话定位

基于 C++、Protobuf、ZooKeeper 与 Reactor TCP 网络库实现分布式 RPC 框架，支持服务注册发现、自定义协议编解码、同步/异步远程调用、请求超时管理、异常请求处理、文件分块传输业务，并通过内存池和压测指标验证高并发短请求与大文件传输场景下的性能表现。

最终应该具备的能力

- 通信层：epoll + 非阻塞 Socket + one loop per thread，网络 IO 与 RPC 业务解耦。
- 协议层：固定长度协议头 + Protobuf 元信息 + 业务消息体，支持半包、粘包和非法包处理。
- 调用层：同步调用、异步调用、request_id 匹配、pending call 管理和超时清理。
- 治理层：基于 ZooKeeper 的服务注册与发现，使用临时节点维护服务实例状态。
- 业务层：FileService 分块上传/下载，用真实业务验证 RPC 框架能力。
- 优化层：内存池/对象池减少高频路径的小对象分配，并用 benchmark 数据对比。
- 交付层：README、架构图、协议文档、错误码表、压测表、面试追问材料。

明确不做的内容

- 不做真正完整的双向 streaming RPC。短期选择“异步接口 + 文件 chunk 分块传输”，更容易实现和讲清楚。
- 不做复杂服务治理，例如熔断、限流、权重负载均衡、灰度发布、链路追踪全套。可以在后续优化中提及。
- 不实现 ZooKeeper/ZAB 本身，只使用 ZooKeeper 做注册中心；ZAB 协议作为面试知识补充。
- 不把内存池写成通用 malloc 替代品，只针对 RPC 高频路径上的 RpcContext、临时编码缓冲区和小块对象复用。

2. 总体架构与模块拆分

整体架构建议采用分层设计：网络层只负责连接和字节流，编解码层负责 frame 拆装，RPC 层负责请求分发与调用管理，业务层只实现具体服务方法。

```
Client Application
|
Generated Stub / RpcClient
|
RpcChannel + PendingCallMap + Timer
|
RpcCodec: FrameHeader + Meta + Payload
|
Reactor TcpClient / TcpConnection
|
===== TCP
=====
|
```

```

Reactor TcpServer / TcpConnection
|
RpcCodec: decode frame, validate protocol
|
RpcServer + ServiceRegistry + Dispatcher
|
Protobuf Service::CallMethod
|
Business Service: EchoService / FileService

```

核心模块表

模块	职责	关键类/文件	面试价值
net	Reactor 网络通信、连接管理、Buffer、定时器	EventLoop, Channel, TcpServer, TcpConnection, Buffer	体现 Linux 网络编程、epoll、非阻塞 IO
codec	协议头编码、解码、半包粘包处理、非法包检测	RpcFrameHeader, RpcCodec, RpcFrame	体现自定义协议设计和健壮性
rpc	服务注册、方法分发、同步/异步调用、pending 管理	RpcServer, RpcClient, RpcChannel, RpcContext	体现 RPC 框架核心能力
registry	服务注册发现、ZooKeeper 临时节点、Watcher 缓存	ZkClient, ServiceDiscovery, ServiceRegistryClient	体现分布式服务治理基础
memory	内存池、对象池、统计指标	FixedBlockPool, ObjectPool, PoolStats	体现性能优化意识
examples	Echo、Calculator、FileService 示例	echo_server, file_server, file_client	让项目不是空框架
benchmark	QPS、延迟、吞吐和资源占用测试	rpc_bench_client, file_transfer_bench	让简历有数据支撑

3. 开发前准备与工程结构

目标是先把工程从“代码混在一起”整理成可以持续升级的结构。第一天不要直接做 FileService 或内存池，先保证基线可编译、可运行、可测试。

推荐目录结构

```

rpc_project/
├── CMakeLists.txt
├── include/
│   ├── net/           # Reactor 网络库
│   ├── codec/         # RpcFrameHeader / RpcCodec
│   ├── rpc/           # RpcServer / RpcClient / RpcChannel
│   ├── registry/      # ZooKeeper 注册发现
│   ├── memory/        # 内存池 / 对象池
│   └── common/        # 日志、错误码、工具函数
├── src/
│   ├── net/
│   ├── codec/
│   ├── rpc/
│   ├── registry/
│   ├── memory/
│   └── common/
├── proto/
└── rpc_meta.proto

```

```

├── file_service.proto
├── examples/
│   ├── echo_server/
│   ├── echo_client/
│   ├── file_server/
│   └── file_client/
├── benchmark/
│   ├── rpc_bench_client.cpp
│   └── file_transfer_bench.cpp
├── tests/
└── docs/

```

依赖与环境

依赖	用途	最低要求
C++17	项目主语言，支持智能指针、lambda、移动语义、线程库	g++ 9+ 或 clang++ 10+
CMake	构建工程、组织 proto 生成代码与多目录模块	能写基本 target_link_libraries
Protobuf	RPC 元信息和业务消息序列化	会使用 protoc、Message、Service 反射
ZooKeeper C API	服务注册发现	能创建临时节点、查询子节点、Watcher
Glog 或自定义日志	记录 RPC 调用链路和异常路径	INFO/WARN/ERROR 级别即可
GTest 可选	单元测试 codec 和错误处理	没有也可先写手动测试
gdb/valgrind/tcpdump	调试崩溃、内存泄漏、网络包	掌握常用命令即可

4. Milestone 1：可运行基线与网络层解耦

目标：先建立一个不会被后续改造破坏的基线版本。这个阶段只要求 Echo 或 Add 这种简单 RPC 能跑通，不做异步、不做 ZooKeeper 复杂缓存、不做内存池。

要做什么

- 整理 CMakeLists.txt，使 net、codec、rpc、examples 可以独立编译。
- 保留现有 Reactor 网络库能力：EventLoop、Channel、TcpServer、TcpConnection、Buffer。
- 确认 EchoServer/EchoClient 或 CalculatorServer/Client 能正常运行。
- 把网络层回调改成只向上抛出“字节流数据”，不要在 TcpConnection 中直接处理 RPC 语义。
- 补充最小日志：连接建立、连接关闭、收到请求、发送响应、错误返回。

验收标准

- 项目可以一键 cmake build。
- examples/echo_server 和 examples/echo_client 可以运行。
- 网络层代码和 RPC 层代码分离。
- 你能口述：TcpConnection 收到数据后如何进入 RpcCodec，再进入 RpcServer 分发。

5. Milestone 2：自定义协议与 RpcCodec

目标：把“长度字段 + 消息体”的简单协议升级为可检查、可扩展、可支持异步匹配的协议。

协议头设计

```
struct RpcFrameHeader {
    uint32_t magic;           // 协议标识，例如 0x20260626
    uint16_t version;        // 协议版本
    uint16_t header_len;     // 固定协议头长度
    uint32_t body_len;       // 后续 body 长度
    uint64_t request_id;     // 请求 ID，用于请求响应匹配
    uint16_t msg_type;       // request / response / heartbeat
    uint16_t status_code;    // OK / error code
    uint32_t checksum;       // 可选：body 校验，第一版可先置 0
};
```

Body 格式建议

| RpcFrameHeader | meta_len | RpcRequestMeta / RpcResponseMeta | user_payload |

RpcFrameHeader: 负责网络层完整性和基础合法性。

meta_len: 表示框架元信息长度。

RpcMeta: 表示 service_name、method_name、request_id、status_code 等 RPC 语义。

user_payload: 业务 Protobuf request/response 的序列化结果。

RpcCodec 接口

```
class RpcCodec {
public:
    std::string encode(const RpcFrameHeader& header,
                      const std::string& body);

    DecodeResult decode(Buffer* buffer, RpcFrame* frame);
};

enum class DecodeStatus {
    OK,
    NEED_MORE_DATA,
    BAD_MAGIC,
    UNSUPPORTED_VERSION,
    FRAME_TOO_LARGE,
    BAD_FRAME
};
```

具体步骤

1. 定义 RpcFrameHeader 和常量：magic、version、max_body_size。
2. 实现网络字节序转换，所有多字节整数进入网络前使用 htonl/htons 或自定义 64 位转换。
3. 实现 encode：把 header 和 body 拼接成连续字节流。
4. 实现 decode：如果 buffer 中不足一个 header，返回 NEED_MORE_DATA；如果 header 合法但 body 未完整，也返回 NEED_MORE_DATA。
5. 支持粘包：decode 成功后从 Buffer 取走一个完整 frame，然后继续循环解析下一个 frame。
6. 支持非法包：magic 错误、版本错误、body_len 超限时返回错误状态，由上层决定返回错误或关闭连接。

必须写的测试

- 完整包解析
- 半包解析
- 一个 Buffer 中包含两个包
- 非法 magic
- body_len 超过限制
- 错误版本号

6. Milestone 3: Protobuf 元信息与服务端分发

目标：让服务端不依赖手写 if/else 分发方法，而是基于 Protobuf Service 反射机制，根据 service_name 和 method_name 找到真正的业务方法。

rpc_meta.proto

```
syntax = "proto3";

package rpc;

message RpcRequestMeta {
    string service_name = 1;
    string method_name = 2;
    uint64 request_id = 3;
    uint32 timeout_ms = 4;
}

message RpcResponseMeta {
    uint64 request_id = 1;
    int32 status_code = 2;
    string error_msg = 3;
}
```

服务端分发流程

```
TcpConnection 收到字节流
    ↓
RpcCodec 解析完整 frame
    ↓
拆出 RpcRequestMeta 和 user_payload
    ↓
根据 service_name 查找 Service
    ↓
根据 method_name 查找 MethodDescriptor
    ↓
创建 request / response Message
    ↓
request->ParseFromString(user_payload)
    ↓
service->CallMethod(method, controller, request, response, done)
    ↓
序列化 response，封装 RpcResponseMeta，返回客户端
```

核心伪代码

```
void RpcServer::onRpcRequest(const RpcRequestMeta& meta,
                             const std::string& payload,
                             const TcpConnectionPtr& conn) {
    auto* service = service_registry_.findService(meta.service_name());
    if (!service) {
        sendError(conn, meta.request_id(), SERVICE_NOT_FOUND);
        return;
    }

    auto* method = service->GetDescriptor()->FindMethodByName(meta.method_name());
    if (!method) {
        sendError(conn, meta.request_id(), METHOD_NOT_FOUND);
        return;
    }

    std::unique_ptr<google::protobuf::Message> request(
        service->GetRequestPrototype(method).New());
    std::unique_ptr<google::protobuf::Message> response(
        service->GetResponsePrototype(method).New());

    if (!request->ParseFromString(payload)) {
        sendError(conn, meta.request_id(), PROTOBUF_PARSE_ERROR);
        return;
    }

    service->CallMethod(method, nullptr, request.get(), response.get(), done);
}
```

验收标准

- 服务端可以注册多个 Protobuf Service。
- 新增 Service 时不需要修改 RpcServer 核心分发代码。
- service 不存在、method 不存在、payload 解析失败都能返回明确错误。
- 能解释 Protobuf Service、MethodDescriptor、Message 和 CallMethod 的关系。

7. Milestone 4: 客户端同步调用

目标：客户端可以像调用本地函数一样发起 RPC，并在收到响应或超时时返回。

同步调用 API 示例

```
RpcClient client("127.0.0.1", 9000);

AddRequest req;
AddResponse resp;
req.set_a(1);
req.set_b(2);

RpcStatus status = client.call("CalculatorService", "Add", req, &resp, 3000);
```


PendingCall 结构

```
struct PendingCall {
    std::mutex mtx;
    std::condition_variable cv;
    bool done = false;
    RpcStatus status;
    std::string response_payload;
};

std::unordered_map<uint64_t, std::shared_ptr<PendingCall>> pending_calls_;
```

同步调用流程

```
生成 request_id
    ↓
构造 RpcRequestMeta
    ↓
序列化 meta + request payload
    ↓
放入 pending_calls_[request_id]
    ↓
发送 frame
    ↓
阻塞等待 condition_variable
    ↓
收到 response 后按 request_id 找到 PendingCall
    ↓
填充 response_payload 醒等待程
    ↓
反序列化 response，返回 RpcStatus
```

验收标准

- Echo/Add 同步调用成功。
- 服务端错误可以通过 RpcStatus 返回给客户端。
- 请求超时后可以从 pending map 中清理，不产生泄漏。
- 响应乱序时也能通过 request_id 找到正确请求。

8. Milestone 5：异步调用、request_id 与超时管理

目标：实现异步 RPC，支持客户端一次发出多个请求，不阻塞当前线程，响应回来后通过 callback 处理。这个阶段是项目从“普通 RPC demo”升级为“工程化 RPC 框架”的关键。

异步接口设计

```
using RpcCallback = std::function<void(const RpcStatus&, const std::string&)>;

uint64_t RpcClient::callAsync(const std::string& service,
                              const std::string& method,
                              const google::protobuf::Message& request,
                              RpcCallback cb,
                              int timeout_ms);
```

异步调用流程

```
callAsync 生成 request_id
↓
构造 PendingAsyncCall{callback, deadline, response_type}
↓
入 pending_async_calls_[request_id]
↓
发送请求 frame
↓
立即返回 request_id
↓
收到 response 根据 request_id 查找 callback
↓
执行 callback 并从 pending map 删除
```

超时管理

- 每个 PendingCall 记录 deadline。
- 使用 EventLoop 定时任务或 timerfd 定期扫描 pending map。
- 超过 deadline 的请求触发 TIMEOUT 回调，并从 map 删除。
- 如果超时后服务端迟到响应，客户端发现 request_id 不存在，直接丢弃或记录 warn 日志。

验收标准

- 客户端可以连续发出 1000 个异步 Echo 请求。
- 响应乱序返回时 callback 仍然对应正确 request_id。
- 服务端延迟响应时客户端能触发 TIMEOUT。
- pending_async_calls_ 在压测结束后为空。

9. Milestone 6: ZooKeeper 服务注册与发现

目标：服务端启动时把服务实例注册到 ZooKeeper，客户端根据 service/method 动态发现可用节点。ZooKeeper 不参与每次 RPC 调用，只在调用前提供服务地址。

推荐节点路径

```
/rpc
  /CalculatorService
    /Add
      /127.0.0.1:9000 # 临时节点
      /127.0.0.1:9001
  /FileService
    /UploadChunk
      /127.0.0.1:9100
```

服务端启动流程

```
RpcServer 启动
↓
```

```
读取 ip、port、service_name、method_name
    ↓
连接 ZooKeeper
    ↓
创建 service/method 路径
    ↓
为当前服务实例创建临时节点
    ↓
开始监听 TCP 端口
```

客户端发现流程

```
客户端用 service.method
    ↓
查询 /rpc/service/method 子节点
    ↓
取可用 ip:port 列表
    ↓
使用随机或轮询选择一个节点
    ↓
建立或复用 TCP 连接
    ↓
发起 RPC 用
```

实现步骤

- 封装 ZkClient: connect、createPath、createEphemeralNode、getChildren、watchChildren。
- 服务端启动时注册所有已注册 Service 的所有 method。
- 客户端 ServiceDiscovery 维护本地服务地址缓存。
- Watcher 收到节点变化后刷新缓存。
- 负载均衡先做随机或轮询，暂时不做权重。

验收标准

- 服务端启动后 ZooKeeper 中能看到临时节点。
- 服务端异常退出后临时节点自动消失。
- 客户端可以不写死 ip:port，而是通过 service/method 发现节点。
- 节点变化后客户端缓存可以更新。

10. Milestone 7：协议错误处理与异常请求防护

目标：处理“不符合协议规范的请求”。这里的“伪造请求”不需要先上升到复杂安全攻防，可以从协议健壮性角度实现：非法 magic、版本不支持、body_len 超限、Protobuf 解析失败、未知服务方法、半包长期不完整等。

错误码设计

```
enum class RpcErrorCode {
    OK = 0,
    BAD_MAGIC = 1001,
    UNSUPPORTED_VERSION = 1002,
```

```

    FRAME_TOO_LARGE = 1003,
    BAD_FRAME = 1004,
    CHECKSUM_ERROR = 1005,
    PROTOBUF_PARSE_ERROR = 1006,
    SERVICE_NOT_FOUND = 1007,
    METHOD_NOT_FOUND = 1008,
    TIMEOUT = 1009,
    INTERNAL_ERROR = 1010
};

```

异常场景	检测位置	处理方式	面试讲法
magic 不匹配	RpcCodec 解析 header	关闭连接	不是本协议的请求，直接断开防止污染状态
version 不支持	RpcCodec	返回错误或关闭连接	协议演进时需要版本兼容判断
body_len 超过上限	RpcCodec	关闭连接	避免大包攻击或异常内存占用
header_len 异常	RpcCodec	关闭连接	防止解析越界
Protobuf 解析失败	RpcServer 分发前	返回 PROTOBUF_PARSE_ERROR	说明业务 payload 不合法
service/method 不存在	ServiceRegistry	返回 SERVICE_NOT_FOUND/METHOD_NOT_FOUND	框架层错误不应导致服务崩溃
请求超时	客户端 Timer	返回 TIMEOUT	避免调用方无限等待
半包长期不完整	连接状态管理	关闭连接	避免慢速发送占用连接资源

必须做的测试

- 构造非法 magic 请求，服务端关闭连接。
- 构造超大 body_len，请求被拒绝。
- 构造错误 Protobuf body，客户端收到 PROTOBUF_PARSE_ERROR。
- 请求不存在的方法，客户端收到 METHOD_NOT_FOUND。
- 服务端故意 sleep，客户端触发 TIMEOUT。

11. Milestone 8：文件分块传输业务

目标：给 RPC 框架增加一个真实业务场景。建议做“异步 RPC + 大文件 chunk 分块上传/下载”，不要做完整 streaming RPC。这样既能体现业务，又能自然引出异步调用、request_id、错误处理、吞吐压测和内存优化。

file_service.proto

```

syntax = "proto3";

package file;

service FileService {
    rpc UploadChunk(UploadChunkRequest) returns (UploadChunkResponse);
    rpc CommitFile(CommitFileRequest) returns (CommitFileResponse);
    rpc DownloadChunk(DownloadChunkRequest) returns (DownloadChunkResponse);
    rpc GetFileMeta(GetFileMetaRequest) returns (GetFileMetaResponse);
}

```

```

message UploadChunkRequest {
    string file_id = 1;
    string filename = 2;
    uint32 chunk_id = 3;
    uint32 total_chunks = 4;
    bytes data = 5;
    uint32 checksum = 6;
}

message UploadChunkResponse {
    bool success = 1;
    string error_msg = 2;
}

message CommitFileRequest {
    string file_id = 1;
    uint32 total_chunks = 2;
    uint64 file_size = 3;
    uint32 checksum = 4;
}

message CommitFileResponse {
    bool success = 1;
    string file_path = 2;
    string error_msg = 3;
}

message DownloadChunkRequest {
    string file_id = 1;
    uint32 chunk_id = 2;
}

message DownloadChunkResponse {
    bool success = 1;
    bytes data = 2;
    bool eof = 3;
    string error_msg = 4;
}

message GetFileMetaRequest {
    string file_id = 1;
}

message GetFileMetaResponse {
    bool exists = 1;
    string filename = 2;
    uint64 file_size = 3;
    uint32 total_chunks = 4;
}

```

上传流程

客户端读取本地文件

↓

按 64KB 或 256KB 切块

↓

```
为每个 chunk 构造 UploadChunkRequest
    ↓
使用 callAsync 并发上传多个 chunk
    ↓
成功 chunk 数和失败 chunk 数
    ↓
全部成功后用 CommitFile
    ↓
服务端合并 chunk，校验 file_size/checksum
```

服务端存储结构

```
storage/
├── tmp/
│   └── {file_id}/
│       ├── 000000.part
│       ├── 000001.part
│       └── ...
└── files/
    └── {file_id}_{filename}
```

实现步骤

- 实现 FileServiceImpl，先支持单线程顺序上传。
- 客户端支持读取文件并切分 chunk。
- 改用 callAsync 并发上传 chunk。
- 服务端按 file_id/chunk_id 写入临时目录。
- CommitFile 检查所有 chunk 是否存在，再按 chunk_id 合并。
- 下载接口按 chunk_id 返回数据，客户端重新拼接并校验 hash。
- 补充错误处理：chunk_id 越界、total_chunks 不匹配、文件不存在、checksum 错误。

验收标准

- 可以上传 10MB 文件。
- 可以上传 100MB 文件。
- 上传完成后服务端文件 hash 与客户端原文件一致。
- 支持并发上传 chunk。
- 某个 chunk 失败时客户端能感知错误。

12. Milestone 9：内存池与对象复用优化

目标：降低 RPC 高频路径中的小对象和临时缓冲区频繁分配释放。不要把内存池宣传成替代 malloc 的通用分配器，而是明确限定在 RpcContext、PendingCall、编解码临时 buffer 等对象。

第一版 FixedBlockPool

```
class FixedBlockPool {
public:
    explicit FixedBlockPool(size_t block_size, size_t block_count);
```

```
void* allocate();
void deallocate(void* ptr);

PoolStats stats() const;

private:
    size_t block_size_;
    std::vector<char> memory_;
    void* free_list_;
    mutable std::mutex mutex_;
};
```

分级池建议

池类型	块大小	适用对象	超过范围处理
SmallPool	4KB	小请求 body、RpcContext、短响应	进入 MediumPool 或 malloc
MediumPool	16KB	中等请求、序列化临时缓冲区	进入 LargePool 或 malloc
LargePool	64KB	文件 chunk 或大响应片段	超过 64KB 走 malloc/new

接入策略

- 先接入 RpcContext / PendingCall 对象池，风险最低。
- 再接入 RpcCodec 序列化临时 buffer。
- 文件 chunk 不强行全部池化，超过阈值走 malloc，避免大块长期占用。
- 提供开关：--use_memory_pool=0/1，方便压测对比。
- 记录统计指标：allocate 总次数、pool hit 次数、fallback 次数、复用率。

验收标准

- 内存池开关可控。
- valgrind 检查无明显内存泄漏。
- 压测表中有开启/关闭内存池对比。
- 能够解释为什么内存池只优化小对象和高频路径。

13. Milestone 10：性能压测与指标体系

目标：用数据证明项目升级有效。即使数据不夸张，也比没有数据强。重点不是跑出特别高的 QPS，而是说明你知道怎么设计实验、怎么解释瓶颈。

短请求 benchmark

```
./rpc_bench_client \
--server=127.0.0.1:9000 \
--concurrency=100 \
--requests=100000 \
--body_size=128 \
--threads=4
```

文件传输 benchmark

```
./file_transfer_bench \  
--server=127.0.0.1:9100 \  
--file=/tmp/test_100mb.bin \  
--chunk_size=65536 \  
--concurrency=32
```

指标定义

指标	含义	怎么计算/记录
QPS	每秒完成 RPC 调用数量	成功请求数 / 总耗时
Avg Latency	平均请求延迟	总延迟 / 成功请求数
P50/P95/P99	延迟分位数	收集每个请求耗时后排序统计
Error Rate	错误率	失败请求数 / 总请求数
Throughput	文件传输吞吐	文件大小 / 上传下载耗时
CPU/Memory	资源占用	top、pidstat、/proc 或日志采样
Pool Hit Rate	内存池命中率	pool_hit / allocate_total

结果表模板

场景	并发	Body/ Chunk	QPS/吞吐	Avg	P95	P99	错误率
Echo	100	128B	待填	待填	待填	待填	待填
Echo	500	128B	待填	待填	待填	待填	待填
UploadChunk	32	64KB	待填 MB/s	待填	待填	待填	待填
内存池关闭	100	128B	待填	待填	待填	待填	待填
内存池开启	100	128B	待填	待填	待填	待填	待填

压测分析要回答的问题

- 为什么并发升高后 P99 延迟上升？
- 瓶颈是在网络 IO、业务处理、锁竞争、Protobuf 序列化，还是磁盘写入？
- 内存池主要改善了 QPS、平均延迟还是尾延迟？为什么？
- 文件传输吞吐受 chunk_size 和并发数影响如何？
- 如果要继续优化，下一步是减少拷贝、连接复用、批量发送，还是改磁盘写入策略？

14. Milestone 11：文档、简历与面试材料

目标：项目完成后，要把代码能力转化成简历材料和面试表达。没有 README、架构图和压测表，项目深度会被削弱。

README 推荐结构

```
# C++ RPC Framework  
  
## 1. 项目简介  
## 2. 核心特性  
## 3. 总体架构
```



```

## 4. RPC 调用流程
## 5. 自定义协议格式
## 6. Protobuf 服务分发机制
## 7. 同步调用
## 8. ZooKeeper 服务注册与发现
## 9. 异常请求处理与错误码
## 10. FileService 文件分块传输
## 11. 内存池优化
## 12. 性能测试结果
## 13. 构建与运行
## 14. 后续优化方向

```

最终简历可写内容方向

维度	简历表达方向
项目描述	基于 C++、Protobuf、ZooKeeper 与 Reactor TCP 网络库实现分布式 RPC 框架，支持服务注册发现、自定义协议、同步/异步调用、文件分块传输和性能压测。
协议设计	设计固定长度协议头 + Protobuf 元信息 + 业务消息体的通信协议，处理 TCP 半包粘包，并支持 request_id 匹配乱序响应。
异步调用	基于 pending map、回调函数和定时器实现异步 RPC 调用与超时清理。
错误处理	完善 magic、version、body_len、Protobuf 解析、未知服务方法、请求超时等异常场景处理。
业务验证	实现 FileService 分块上传/下载，支持大文件按 chunk 并发传输、commit 校验和错误返回。
性能优化	实现简化内存池/对象池复用 RpcContext 与临时缓冲区，并通过 QPS、P95/P99、吞吐等指标对比优化效果。

面试必须准备的问题

- 为什么使用固定长度协议头 + Protobuf 消息体？
- TCP 粘包半包怎么处理？Buffer 中有多个包怎么办？
- request_id 如何支持异步调用和响应乱序？
- 服务端如何基于 Protobuf 反射调用方法？
- ZooKeeper 在 RPC 框架里负责什么？为什么不参与每次调用？
- 伪造请求或非法请求怎么处理？
- 内存池优化了什么？为什么不能替代 malloc？
- FileService 为什么能体现 RPC 框架能力？
- 你的 RPC 和 brpc/gRPC 相比差距在哪里？
- 压测结果说明了什么？瓶颈在哪里？

15. 推荐时间线与优先级

如果时间有限，按 P0/P1/P2 推进。P0 完成后项目就可以写入简历；P1 会显著增强项目深度；P2 是加分项。

阶段	时间建议	任务	优先级
阶段 1	第 1-2 天	整理工程结构，跑通 Echo/Add 基线	P0
阶段 2	第 3-5 天	实现 RpcFrameHeader、	P0

		RpcCodec、半包粘包处理和非法包测试	
阶段 3	第 6-8 天	实现 RpcRequestMeta/RpcResponseMeta、服务端注册与分发	P0
阶段 4	第 9-10 天	实现客户端同步调用、PendingCall、超时返回	P0
阶段 5	第 11-13 天	实现异步调用、callback、request_id 乱序匹配、超时清理	P0
阶段 6	第 14-15 天	接入 ZooKeeper 注册发现和本地缓存	P1
阶段 7	第 16-18 天	补齐错误码、异常请求处理、非法包测试	P0
阶段 8	第 19-22 天	实现 FileService 分块上传/下载	P1
阶段 9	第 23-25 天	实现内存池/对象池和统计指标	P1
阶段 10	第 26-28 天	写 benchmark，产出 QPS/P95/P99/吞吐数据	P0
阶段 11	第 29-30 天	整理 README、架构图、简历描述和面试问题	P0

压缩版本

如果只有两周，优先完成：协议头 + RpcCodec + 服务分发 + 同步/异步调用 + 错误处理 + benchmark。FileService 和内存池可以做最小版。如果只有一周，至少完成协议健壮性和异步调用，因为这两个最能和原始教程型 RPC 拉开差距。

16. 知识准备清单

RPC 项目需要补的知识不要无限扩张。以下按面试价值排序。

知识点	必须掌握	掌握到什么程度
Protobuf wire format	field number、wire type、varint、length-delimited、tag 编码	能解释为什么 Protobuf 适合 RPC，能看懂二进制编码的大致结构
Protobuf 反射	Service、Message、MethodDescriptor、CallMethod	能解释服务端如何根据字符串分发方法
TCP 粘包半包	字节流语义、Buffer 累积、长度字段拆包	能结合 RpcCodec 讲清楚
epoll/Reactor	非阻塞 IO、事件回调、one loop per thread	能结合网络层解释
异步编程	callback、pending map、request_id、超时	能解释响应乱序如何处理
ZooKeeper	临时节点、Watcher、服务注册发现	能解释客户端缓存和节点变化
ZAB 协议	Leader、Follower、事务提议、过半确认	了解原理即可，不需要自己实现
brpc 文档	异步调用、超时、错误码、内置服务、连接复用	用于对标企业级 RPC，不要求复现
内存池	固定块、空闲链表、对象池、线程局部缓存	能解释优化对象和限制
Linux 调试工具	gdb、valgrind、tcpdump、Wireshark	能定位崩溃、泄漏和网络收发问题

17. 最终验收清单

代码功能验收

- Echo/Add 同步 RPC 调用成功。
- 异步 RPC 支持多个并发请求，响应乱序也能正确回调。
- 请求超时可以正确返回 TIMEOUT，并清理 pending map。
- ZooKeeper 注册发现可用，服务端退出后临时节点消失。
- FileService 可以上传/下载 10MB 和 100MB 文件，hash 一致。
- 非法 magic、超大 body、错误 Protobuf、未知方法都有明确处理。
- 内存池开启/关闭均可运行，无明显泄漏。

性能与文档验收

- 有短请求 QPS、Avg、P95、P99、错误率表格。
- 有文件传输吞吐 MB/s 表格。
- 有内存池开启/关闭对比数据。
- README 包含架构、协议、运行方式和 benchmark。
- docs 中有协议格式说明和错误码表。
- 简历中的每一句技术描述都有代码或实验支撑。

面试表达验收

- 能 3 分钟讲清楚项目背景、架构、核心流程和难点。
- 能画出客户端调用流程、服务端分发流程和协议格式。
- 能回答“这个项目和普通 muduo/RPC 教程有什么区别”。
- 能说明 brpc/gRPC 比你的项目多了哪些企业级能力。
- 能根据压测结果说出瓶颈和后续优化方向。

18. 第一天可执行任务

第一天不要写 FileService，也不要先写内存池。先把协议和调用主线定下来。

任务	具体动作	完成标准
画 RPC 调用流程	画客户端、网络层、编解码、服务端分发、业务方法、响应返回链路	能口述一次完整调用流程
定义 RpcFrameHeader	写 header 字段、常量、网络字节序转换函数	能 encode/decode header
定义 RpcErrorCode	列出 BAD_MAGIC、FRAME_TOO_LARGE、TIMEOUT 等错误码	错误码表初版完成
定义 rpc_meta.proto	写 RpcRequestMeta 和 RpcResponseMeta	protoc 能生成代码
设计 RpcCodec 接口	写 encode/decode 接口和 DecodeStatus	接口文件完成
写测试计划	列出完整包、半包、粘包、非法 magic 测试	至少手动跑通一个完整包解析

第一天结束时，你应该已经从“要升级 RPC”变成“知道协议、元信息、编解码和调用链怎么落地”。后续每一天都围绕这个主线向外扩展。